Plataforma Inteligente de Mantenimiento Predictivo

Estudiante

Daniel Camilo Ramos Garcia

Docente

Eliecer Montero Ojeda

Unidades Tecnológicas de Santander

Ingeniería de Sistemas

Bucaramanga

2026

Patrón Bridge

El patrón Bridge permite separar una abstracción de su implementación, de modo que ambas puedan variar de manera independiente.

Este patrón resulta útil cuando un sistema necesita combinar diferentes tipos de objetos con distintas formas de ejecución, evitando una explosión de clases y reduciendo el acoplamiento entre componentes.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Bridge se utilizará para desacoplar el tipo de alerta de su canal de notificación.

Esto significa que el sistema puede manejar diferentes tipos de alertas, por ejemplo:

- alerta crítica
- alerta preventiva

y enviarlas a través de diferentes canales, por ejemplo:

- correo electrónico
- panel del sistema

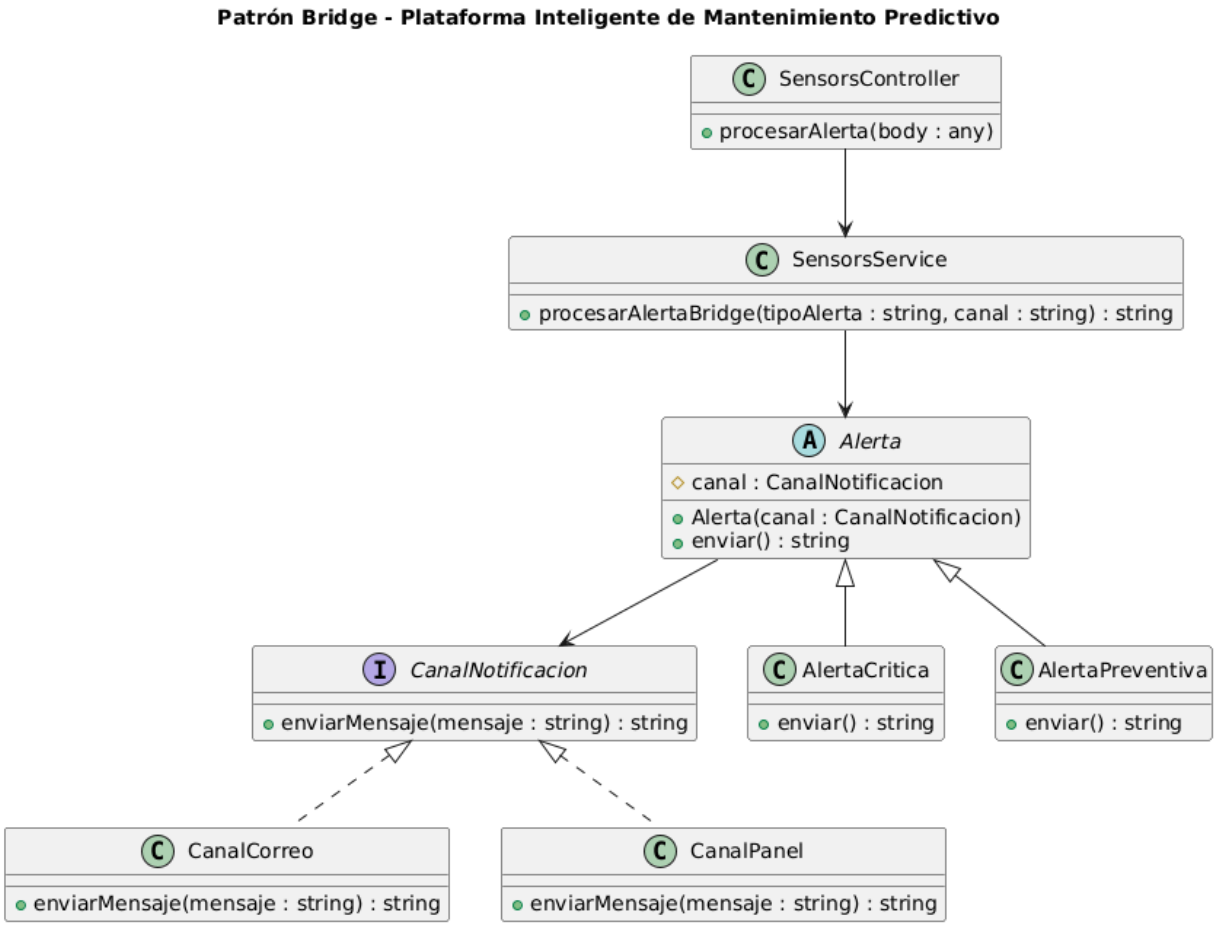
De esta manera, el sistema puede combinar libremente cada tipo de alerta con cualquier canal de notificación sin depender de clases concretas.

Ventajas de aplicar este patrón

- Permite desacoplar la lógica de la alerta de la forma en que se notifica.
- Facilita agregar nuevos tipos de alertas.

- Facilita agregar nuevos canales de notificación.
- Mejora la escalabilidad y mantenibilidad del sistema.
- Reduce el acoplamiento entre componentes del backend

Diagrama UML del patrón Bridge



Implementación del patrón Bridge

Primero se creó la interfaz que representa la implementación del canal de notificación.

Archivo

src/alerts/bridge/canal-notificacion.interface.ts

```
TS canal-notificacion.interface.ts U ×
src > alerts > bridge > TS canal-notificacion.interface.ts > CanalNotificacion
1  export interface CanalNotificacion {
2    enviarMensaje(mensaje: string): string;
3  }
```

Implementaciones concretas de canal

Se implementaron dos canales concretos para enviar alertas dentro del sistema.

src/alerts/bridge/canal-correo.ts

```
src > alerts > bridge > TS canal-correo.ts > CanalCorreo
1  import { CanalNotificacion } from './canal-notificacion.interface';
2
3  export class CanalCorreo implements CanalNotificacion {
4    enviarMensaje(mensaje: string): string {
5      return `Correo enviado con mensaje: ${mensaje}`;
6    }
7  }
```

Canal del panel del sistema

src/alerts/bridge/canal-panel.ts

Abstracción principal

Posteriormente se creó la abstracción Alerta, la cual trabaja con cualquier implementación de canal de notificación.

Archivo

src/alerts/bridge/alerta.abstract.ts

src > alerts > bridge > TS alerta.abstract.ts > Alerta

```
1  import { CanalNotificacion } from './canal-notificacion.interface';
2
3  export abstract class Alerta {
4    constructor(protected canal: CanalNotificacion) {}
5
6    abstract enviar(): string;
7  }
```

Alertas concretas

Se implementaron dos tipos de alertas dentro del sistema.

Alerta crítica

src/alerts/bridge/alerta-critica.ts

```
src > alerts > bridge > TS alerta-critica.ts > AlertaCritica
import { Alerta } from './alerta.abstract';
import { CanalNotificacion } from './canal-notificacion.interface';

export class AlertaCritica extends Alerta {
  constructor(canal: CanalNotificacion) {
    super(canal);
  }

  enviar(): string {
    return this.canal.enviarMensaje('ALERTA CRÍTICA: posible falla grave detectada');
  }
}
```

Alerta preventiva

src/alerts/bridge/alerta-preventiva.ts

```
src > alerts > bridge > TS alerta-preventiva.ts > AlertaPreventiva
import { Alerta } from './alerta.abstract';
import { CanalNotificacion } from './canal-notificacion.interface';

export class AlertaPreventiva extends Alerta {
  constructor(canal: CanalNotificacion) {
    super(canal);
  }

  enviar(): string {
    return this.canal.enviarMensaje('ALERTA PREVENTIVA: revisar comportamiento del activo');
  }
}
```

Integración del patrón en el sistema

Para integrar el patrón Bridge se implementó un método en el servicio de sensores que recibe el tipo de alerta y el canal deseado, construye los objetos adecuados y ejecuta el envío.

Archivo

src/sensors/sensors.service.ts

```
//PATRON BRIDGE

procesarAlertaBridge(tipoAlerta: string, canal: string) {
  let canalNotificacion;
  let alerta;

  if (canal === 'correo') {
    canalNotificacion = new CanalCorreo();
  } else {
    canalNotificacion = new CanalPanel();
  }

  if (tipoAlerta === 'critica') {
    alerta = new AlertaCritica(canalNotificacion);
  } else {
    alerta = new AlertaPreventiva(canalNotificacion);
  }

  return alerta.enviar();
}
```

Endpoint del controlador

Para probar el patrón Bridge se creó un endpoint que permite seleccionar el tipo de alerta y el canal de notificación.

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT BRIDGE
@Post("alerta-bridge")
procesarAlerta(@Body() body: any) {

  return this.sensorsService.procesarAlertaBridge(body.tipoAlerta, body.canal);

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

POST http://localhost:3000/sensors/alerta-bridge

Prueba 1 — Alerta crítica por correo

Petición enviada

none JSON Text

```
{
  "tipoAlerta": "critica",
  "canal": "correo"
}
```

Respuesta

```
"Correo enviado con mensaje: ALERTA CRÍTICA: posible falla grave detectada"
```

Prueba 2 — Alerta preventiva en panel

Petición enviada

none JSON Text

```
{
  "tipoAlerta": "preventiva",
  "canal": "panel"
}
```

Respuesta

```
"Notificación mostrada en panel: ALERTA PREVENTIVA: revisar comportamiento del activo"
```

Esto confirma que la abstracción Alerta y su implementación CanalNotificacion están desacopladas, tal como lo propone el patrón Bridge.

Patrón Decorator

El patrón Decorator permite agregar nuevas responsabilidades o comportamientos a un objeto de manera dinámica, sin modificar su estructura original.

Este patrón resulta útil cuando se desea extender la funcionalidad de un componente base de forma flexible y reutilizable.

Aplicación en nuestro sistema

En la Plataforma Inteligente de Mantenimiento Predictivo, el patrón Decorator se utilizará para enriquecer el procesamiento de lecturas de sensores.

En este caso, una lectura base de sensor puede ser decorada con funcionalidades adicionales, por ejemplo:

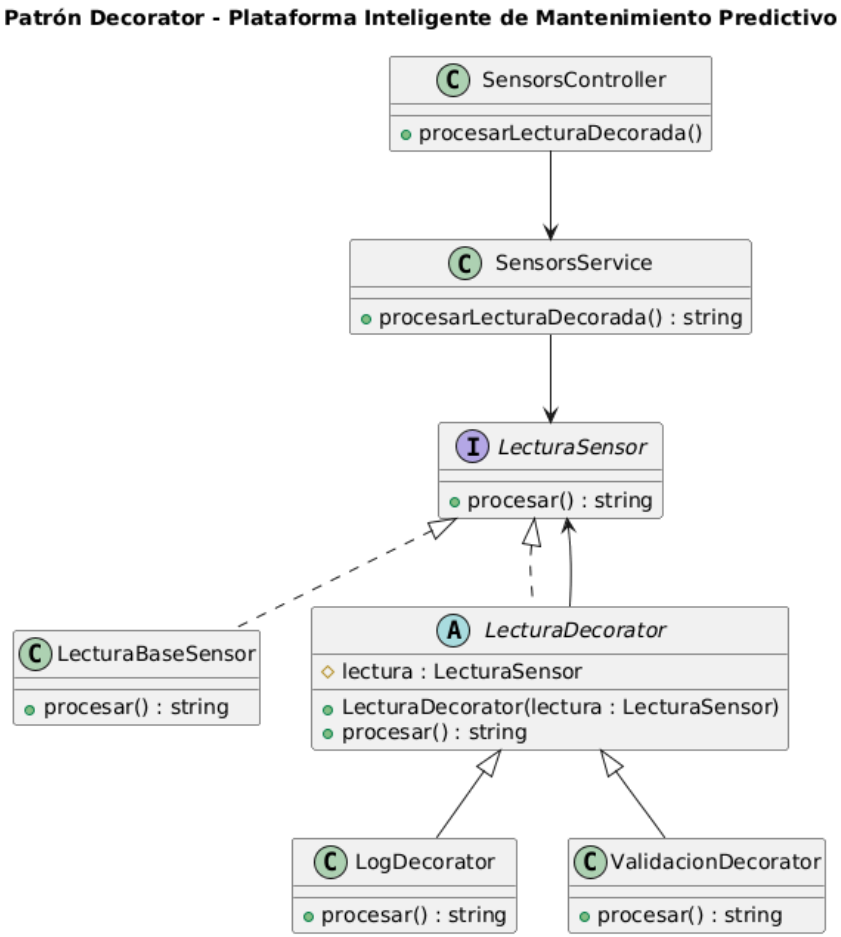
- registro en log
- validación de rangos
- marcado de prioridad

Esto permite ampliar la funcionalidad del procesamiento sin modificar la clase original del sensor.

Ventajas de aplicar este patrón

- Permite extender funcionalidades sin alterar el objeto base.
- Favorece el principio de abierto/cerrado.
- Facilita combinar múltiples comportamientos adicionales.
- Reduce el acoplamiento y mejora la reutilización del código.
- Permite construir flujos de procesamiento más flexibles.

Diagrama UML del patrón Decorator



Implementación del patrón Decorator

Se creó primero una interfaz común para representar el procesamiento de lecturas de sensores.

src > sensors > decorator > TS lectura-sensor.interface.ts > LecturaSensor

```
1  export interface LecturaSensor {
2    procesar(): string;
3  }
```

Componente base

Se implementó una clase base que representa el procesamiento simple de una lectura de sensor.

Archivo

src/sensors/decorator/lectura-base-sensor.ts

src > sensors > decorator > TS lectura-base-sensor.ts > LecturaBaseSensor

```
1  import { LecturaSensor } from './lectura-sensor.interface';
2
3  export class LecturaBaseSensor implements LecturaSensor {
4    procesar(): string {
5      return 'Lectura base del sensor procesada';
6    }
7  }
```

Decorador abstracto

Luego se implementó un decorador abstracto que mantiene la referencia al componente base.

Archivo

src/sensors/decorator/lectura.decorator.ts

src > sensors > decorator > TS lectura.decorator.ts > LecturaDecorator

```
1  import { LecturaSensor } from './lectura-sensor.interface';
2
3  export abstract class LecturaDecorator implements LecturaSensor {
4    constructor(protected lectura: LecturaSensor) {}
5
6    procesar(): string {
7      return this.lectura.procesar();
8    }
9  }
```

Decoradores concretos

Se implementaron decoradores que agregan nuevas funcionalidades al procesamiento de la lectura.

Decorador de log

src/sensors/decorator/log.decorator.ts

src > sensors > decorator > TS log.decorator.ts > LogDecorator

```typescript
import { LecturaDecorator } from './lectura.decorator';
import { LecturaSensor } from './lectura-sensor.interface';

export class LogDecorator extends LecturaDecorator {
  constructor(lectura: LecturaSensor) {
    super(lectura);
  }

  procesar(): string {
    return `${super.procesar()} | Registro en log completado`;
  }
}
```

Decorador de validación

src/sensors/decorator/validacion.decorator.ts

src > sensors > decorator > TS validacion.decorator.ts > ValidacionDecorator

```typescript
import { LecturaDecorator } from './lectura.decorator';
import { LecturaSensor } from './lectura-sensor.interface';

export class ValidacionDecorator extends LecturaDecorator {
  constructor(lectura: LecturaSensor) {
    super(lectura);
  }

  procesar(): string {
    return `${super.procesar()} | Validación de rango ejecutada`;
  }
}
```

Integración del patrón en el sistema

Para integrar el patrón Decorator se creó un método en el servicio de sensores que construye una lectura base y la decora con comportamientos adicionales.

Archivo

src/sensors/sensors.service.ts

```
//PATRON DECORATOR
procesarLecturaDecorada() {
  let lectura = new LecturaBaseSensor();

  lectura = new LogDecorator(lectura);
  lectura = new ValidacionDecorator(lectura);

  return lectura.procesar();
}
```

Endpoint del controlador

Para probar el patrón Decorator se creó un endpoint que ejecuta el procesamiento decorado de una lectura de sensor.

Archivo

src/sensors/sensors.controller.ts

```
// ENDPOINT DECORATOR
@Post("lectura-decorada")
procesarLecturaDecorada() {

  return this.sensorsService.procesarLecturaDecorada();

}
```

Pruebas del sistema

Las pruebas se realizaron utilizando Postman para enviar solicitudes HTTP al backend.

Endpoint utilizado

POST http://localhost:3000/sensors/lectura-decorada

Prueba del sistema

Para probar el patrón Decorator se envió una petición HTTP al endpoint correspondiente.

En este caso no es necesario enviar información en el body, ya que el sistema internamente construye la lectura base y le agrega los decoradores definidos.

Petición enviada

No requiere body.

Respuesta

```
"Lectura base del sensor procesada | Registro en log completado | Validación de rango ejecutada"
```

La respuesta demuestra que el sistema primero ejecutó el procesamiento base y luego fue agregando dinámicamente nuevas responsabilidades mediante los decoradores.

La implementación del patrón Decorator permite agregar funcionalidades al procesamiento de lecturas de sensores sin modificar la estructura base del sistema.